# Projet 2 — Plan Decathlon et répartition

Oct 19, 2026 · @Abdoulaye

## Vue d'ensemble

On construit une petite base de données de produits Decathlon Canada (en français) à partir du site, puis on l'interroge dans CouchDB. Le projet vaut 15 % de la note, dure 3 semaines et se remet dans Léa au cours 9. Le site a déjà été approuvé par l'enseignant.

1. **Scraper** les pages produits de decathlon.ca/fr et enregistrer le résultat brut dans des fichiers CSV (UTF-8, une ligne d'en-têtes). Ces CSV ne sont jamais modifiés après.
2. **Nettoyer et transformer** les CSV en `donnees.json` : doublons retirés, espaces et formats corrigés, nombres convertis, valeurs manquantes traitées.
3. **Importer** le JSON dans une base CouchDB, puis vérifier le nombre de documents, les types et les accents.
4. **Interroger** avec 3 requêtes : 2 en Mango (filtre multicritère avec projection, tri avec limite) et 1 vue MapReduce (regroupement avec calcul), avec les index nécessaires.
5. **Documenter** : `rapport.md`, README de reproduction, dépôt GitHub.

Le critère à ne pas manquer : **au moins 2 500 produits distincts après nettoyage**. La remise doit permettre de recréer la base à partir du JSON, sans refaire le scraping.

| Critère d'évaluation | Points |
|---|---|
| Extraction HTML, CSV bruts et qualité des données | 25 |
| Requêtes et interprétation des résultats | 20 |
| Modélisation des documents | 15 |
| Nettoyage, transformation JSON et importation vérifiée | 15 |
| GitHub, reproductibilité et collaboration | 15 |
| Rapport Markdown | 10 |

# Le site Decathlon : ce qu'on peut faire et ce qui est interdit

Les pages de liste que j'ai consultées affichent déjà dans le HTML le nom du produit, l'URL, la note, le nombre d'avis et le prix, donc `requests` + `BeautifulSoup` devraient suffire. Les détails d'une page produit (marque, catégorie, description, images) restent à confirmer en ouvrant quelques pages.

**Pour trouver les produits** : le `robots.txt` déclare un sitemap de produits (`https://www.decathlon.ca/decathlon-products-index.xml`). C'est la source de la liste d'URLs.

**Interdit par `robots.txt` (on ne le fait pas)**

- Tout ce qui est sous `/api/*`.
- Les URLs avec `sort=`, `direction=`, `notes=`, `reviews_page=`, `reviews_note=` et la plupart des filtres `/f-*` (seuls `f-brand_`, `f-color_` et `f-gender_` sont permis).
- Les chemins `checkout`, `account` et `login`.
- Les pages de recherche (`search?Ntt=`).

**Règles de politesse (exigées par l'énoncé)**

- 2 à 3 secondes entre chaque requête, une requête à la fois.
- Un User-Agent clair qui identifie le projet scolaire.
- Si le site nous bloque (une protection Incapsula est mentionnée), on ralentit ou on s'arrête. On ne la contourne pas.

**Pièges connus**

- Un même produit apparaît sous plusieurs URLs (ex. `306373/c382m8543179`). Il faut dédupliquer sur l'identifiant produit (`306373`) pour atteindre 2 500 produits *distincts*.
- Prix `11,99 $` : virgule décimale et symbole à retirer.
- Avis `(1 907)` : espace dans le nombre à retirer.
- Certains produits n'ont ni note ni avis : valeur manquante (`null`), jamais `0`.

Sources : page d'accueil decathlon.ca/fr, robots.txt (consultés le 2026-10-19).

# Modèle de document proposé (à valider ensemble)

Un document = un produit Decathlon, identifié par son numéro de produit (celui qu'on voit dans l'URL). Les questions auxquelles la base répondra : quels produits d'une catégorie sont bon marché et bien notés, quels sont les plus commentés, et quel est le prix moyen par catégorie.

| Champ | Type JSON | Exemple | Remarque |
|---|---|---|---|
| `_id` | chaîne | `"decathlon-306373"` | unique et stable : préfixe + numéro de produit |
| `nom` | chaîne | `"Mitaines de ski chaudes et imperméables enfant, 100"` | accents conservés, espaces nettoyés |
| `url_source` | chaîne | `"https://www.decathlon.ca/fr/p/.../306373/..."` | permet de retrouver l'origine |
| `categorie` | chaîne | `"Gants de ski"` | pour filtrer et regrouper (à confirmer sur la page produit) |
| `marque` | chaîne ou null | `null` si inconnue | à confirmer sur la page produit |
| `prix` | objet | `{"montant": 11.99, "devise": "CAD"}` | `montant` décimal |
| `evaluation` | objet ou null | `{"note": 4.7, "nb_avis": 1907}` | `null` si le produit n'a aucun avis (jamais 0) |
| `image_url` | chaîne ou null | URL seulement | aucune image téléchargée |
| `date_extraction` | chaîne | `"2026-10-20"` | format ISO AAAA-MM-JJ |

**Conventions** : les dates en ISO 8601, une valeur inconnue = `null` (jamais 0 ni chaîne vide), les nombres sans symbole ni espace. L'objet imbriqué `evaluation` regroupe ce qui se consulte ensemble; les champs `prix.montant`, `evaluation.note` et `evaluation.nb_avis` serviront aux filtres et aux tris.

Exemple de document (le second exemple, avec valeurs manquantes, sera à ajouter au rapport) :

```
{
  "_id": "decathlon-306373",
  "nom": "Mitaines de ski chaudes et imperméables enfant, 100",
  "url_source": "https://www.decathlon.ca/fr/p/mitaines-de-ski-chaudes-et-
impermeables-enfant-100/306373/c382m8543179",
  "categorie": "Gants de ski",
  "marque": null,
  "prix": { "montant": 11.99, "devise": "CAD" },
  "evaluation": { "note": 4.7, "nb_avis": 1907 },
  "image_url": null,
  "date_extraction": "2026-10-20"
}
```

Ce modèle est une proposition de départ. Les champs `categorie`, `marque` et `image_url` dépendent de ce que l'extraction donne réellement : la personne A les confirme après avoir testé quelques pages.

## Étapes concrètes, dans l'ordre

1. **Dépôt GitHub** : créer le dépôt, inviter les 2 coéquipiers et `nicolas.payre@cegepsherbrooke.qc.ca`. Livrable : dépôt avec README initial et `.gitignore`.
2. **Modèle** : valider ensemble les champs et types du document JSON (section précédente). Livrable : schéma écrit dans le README.
3. **Test d'extraction** : ouvrir 5 à 10 pages produits, confirmer où se trouvent catégorie, marque et image dans le HTML. Livrable : liste des sélecteurs retenus.
4. **Liste d'URLs** : lire le sitemap de produits et enregistrer les URLs. Livrable : `urls.txt`.
5. **Scraping complet** : visiter les pages avec 2 à 3 s de délai, écrire les CSV bruts au fur et à mesure (reprise possible si le script plante). Viser **3 000 produits ou plus** pour garder une marge après déduplication. Livrable : `data/raw/*.csv`.
6. **Nettoyage** : lire les CSV, dédupliquer sur le numéro de produit, convertir prix et avis en nombres, mettre `null` aux valeurs manquantes, compter avant et après. Livrable : `data/donnees.json` + nombres pour le rapport.
7. **CouchDB** : créer la base, importer le JSON, vérifier le nombre de documents, les types et les accents. Livrable : script d'import reproductible.
8. **Requêtes** : deux Mango avec leurs index et une vue MapReduce. Exemples d'idées : filtre catégorie + prix inférieur à un seuil + note minimale avec projection de `nom`, `prix`, `evaluation` ; les 10 produits les plus commentés (tri sur `nb_avis` avec limite); prix moyen par catégorie (vue avec `_stats`). Livrable : fichiers de requêtes, index et définition de la vue.

9. **Rapport et README** : `rapport.md` (contenu listé dans la remise) et README pour tout recréer sans refaire le scraping. Livrable : les 2 fichiers.
10. **ZIP final** : vérifier qu'aucun mot de passe ni jeton n'est dans le dépôt, puis déposer dans Léa.

Structure de dépôt suggérée :

```
projet2-decathlon/
  README.md
  rapport.md
  scraper/        # scripts d'extraction
  nettoyage/      # script de nettoyage et transformation
  couchdb/        # import, index Mango, vue MapReduce, requêtes
  data/raw/       # CSV bruts (ne jamais modifier)
  data/donnees.json
```

## Répartition entre les 3 membres

Chaque personne possède un bloc technique et le défend à l'oral (l'énoncé exige que chacun puisse expliquer sa contribution). Remplacez A, B et C par vos noms.

| Personne | Rôle | Tâches (étapes) | Fichiers dont elle est responsable |
|---|---|---|---|
| A : à compléter | Extraction | Test d'extraction (3), liste d'URLs (4), scraping complet (5), confirmation des champs auprès de B | `scraper/`, `data/raw/*.csv` |
| B : à compléter | Modélisation et nettoyage | Modèle JSON (2), nettoyage et transformation (6), comptes avant/après | `nettoyage/`, `data/donnees.json` |
| C : à compléter | CouchDB et requêtes | Installation de CouchDB, import (7), index Mango, 2 requêtes Mango, vue MapReduce (8) | `couchdb/` |

**Partagé par les 3**

- Création du dépôt et invitations (1) : A le crée, B et C l'acceptent.
- Rapport (9) : chacun écrit la section de son bloc; une personne assemble et relit. La contribution et le pourcentage de chacun se remplissent à la fin.

- README (9) : chacun documente comment exécuter son bloc; C vérifie la reproduction complète depuis `donnees.json` sur une machine propre.
- Mention de l'utilisation de l'IA dans le rapport, avec les vérifications faites (relecture du code, test des résultats).

**Règles Git**

- Chacun travaille sur sa branche et fait des commits significatifs et fréquents; on fusionne dans `main` par demande de fusion.
- Les contributions de tous doivent être visibles dans l'historique : un membre sans commit perd des points.
- Aucun mot de passe ni jeton dans le dépôt (identifiants CouchDB dans une variable d'environnement ou un fichier `.env` ignoré par Git, avec un `.env.example`).
- Les CSV bruts ne sont jamais édités à la main.

## Calendrier sur 3 semaines et liste de contrôle de la remise

Les dates de cours 7 et 9 sont à confirmer dans Léa; le plan suppose que le cours 9 tombe vers la fin de la semaine 3. Le site est déjà approuvé.

| Semaine | Objectif | Livrable de fin de semaine |
|---|---|---|
| 1 (du 19 au 25 oct.) | Dépôt créé, modèle validé, test d'extraction, CouchDB installé, scraping lancé | Premiers CSV bruts, schéma du document écrit |
| 2 (du 26 oct. au 1er nov.) | Scraping terminé (3 000 produits ou plus), nettoyage, import dans CouchDB | `donnees.json` avec 2 500 éléments distincts ou plus, base importée et vérifiée |
| 3 (du 2 au 8 nov.) | Requêtes, index, vue MapReduce, rapport, README, test de reproduction | ZIP complet déposé dans Léa |

Si le scraping prend du retard, c'est la seule étape qui bloque tout le reste : il faut le lancer dès la semaine 1, même avec un script imparfait, et le laisser tourner pendant que B et C préparent leurs parties sur un petit échantillon de 50 produits.

**Liste de contrôle de la remise**

- [ ] `rapport.md` : membres, site et outil, nettoyage et comptes avant/après, structure et 2 exemples avec justification, méthode d'import et vérifications, 3 requêtes avec extrait et interprétation, lien GitHub, nom de la base, avis sur les technologies, contribution et pourcentage de chacun

- [ ] CSV bruts (UTF-8, avec en-têtes) et `donnees.json` nettoyé (2 500 éléments distincts ou plus)
- [ ] Scripts d'extraction, de nettoyage et d'import; requêtes, index et définition de la vue
- [ ] README qui recrée la base depuis le JSON, sans refaire le scraping
- [ ] Dépôt GitHub avec le prof invité et des commits de chaque membre
- [ ] Aucun mot de passe ni jeton dans le dépôt ou le ZIP
- [ ] Mention de l'IA et des vérifications dans le rapport
- [ ] Images éventuelles jointes au rapport
- [ ] ZIP déposé dans Léa au cours 9